Qassim University جامعة القصيم

Machine learning project

Machine Learning Engineer Salary Prediction

Analyzing Salaries in the ML Industry

Yazeed Ali Alwehaibi	421108130
Ali Khaled Al Orayni	411107789

Introduction

In this project we analyze and predict salaries for machine learning engineers based on a range of factors such as work experience, job title, location, and more. In this report, we will discuss the algorithms used, model performance metrics, and insights gained from the analysis.

Algorithms used

In this project, we used two algorithms, Linear Regression and Random Forest Regression, to predict salaries of machine learning engineers.

1. **Linear Regression**:
    - **Easy Understanding**: Linear Regression helps us to see how each factor affects the salary in a straightforward way.
    - **Quick Calculations**: It is fast and simple to calculate, making it a good starting point for predictions.
2. **Random Forest Regression**:
    - **Seeing the Big Picture**: Random Forest looks at all factors together to predict salaries, capturing complex patterns that Linear Regression might miss.
    - **Avoiding Mistakes**: It is good at avoiding mistakes like overfitting, which can happen when we rely too heavily on one type of information.

linear regression

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable.

Code used:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

df = pd.read_csv('salaries.csv')

features = ['work_year', 'experience_level', 'employment_type',
            'employee_residence', 'remote_ratio',
'company_location', 'company_size']
target = 'salary_in_usd'

X = df[features]
y = df[target]

# Split the data into training and testing sets (80% train, 20%
test)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

categorical_features = ['experience_level', 'employment_type',
'employee_residence', 'company_location', 'company_size']
numerical_features = ['work_year', 'remote_ratio']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='constant',
fill_value='Unknown')),
        ('encoder', OneHotEncoder(handle_unknown='ignore'))  #
Handle unknown categories
    ]), categorical_features)
])
```

```
model = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Absolute Error (MAE): {mae}')
print(f'Mean Squared Error (MSE): {mse}')
print(f'R-squared (R2): {r2}')

# Calculate and print the average salary
average_salary = df['salary_in_usd'].mean()
print(f'Average Yearly Salary: ${average_salary:.2f}')
```

output:

```
Mean Absolute Error (MAE): 45352.87245056721
Mean Squared Error (MSE): 3887440227.951392
R-squared (R2): 0.2010299806392276
Average Yearly Salary: $149713.58
```

Random forest algorithm

We will use a Random Forest model to estimate salary in this project. The Random Forest is an ensemble learning method that is known for being reliable and accurate. It works by building many decision trees during training and then showing the average forecast of all the trees. The variable we want to focus on is salary_in_usd. It will come from a dataset that has information about jobs, like job title, amount of experience, and company size. We want to find important factors that affect pay and improve the accuracy of our predictions using job data by using this model.

Code used:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline


file_path = 'salaries.csv'
data = pd.read_csv(file_path)


features = ['work_year', 'experience_level', 'employment_type', 'job_title',
  'remote_ratio', 'company_location', 'company_size']  # Add or remove features as necessary
X = data[features]
y = data['salary_in_usd']

categorical_cols = [cname for cname in X.columns if X[cname].dtype == "object"]
numerical_cols = [cname for cname in X.columns if X[cname].dtype in ['int64', 'float64']]


categorical_transformer = OneHotEncoder(handle_unknown='ignore')


preprocessor = ColumnTransformer(
    transformers=[
        ('cat', categorical_transformer, categorical_cols),
        ('num', 'passthrough', numerical_cols)
    ])
```

```
model = RandomForestRegressor(n_estimators=100, random_state=0)


pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                           ('model', model)])


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)


pipeline.fit(X_train, y_train)


y_pred = pipeline.predict(X_test)


mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Mean Absolute Error:", mae)
print("R-squared:", r2)
```

Output:

```
Mean Squared Error: 3625851220.286806
Mean Absolute Error: 42372.811368702496
R-squared: 0.2561633816794977
```

A few adjustments have been made to the parameters resulting in a slight improvement

```
model = RandomForestRegressor(
  n_estimators=100,
  max_depth=30,
  min_samples_split=8,
  min_samples_leaf=5,
  random_state=0)
```

output:

```
Mean Squared Error: 3355600048.7951174
Mean Absolute Error: 41019.53861148382
R-squared: 0.31160490569316934
```

Comparison

1. **Mean Absolute Error (MAE)**:
    - Random Forest Regression has a lower MAE ($41,019.54) compared to Linear Regression ($45,352.87), indicating better accuracy in predicting salaries.
2. **Mean Squared Error (MSE)**:
    - Random Forest Regression also has a lower MSE ($3,355,600,048.80) compared to Linear Regression ($3,887,440,227.95), again indicating better accuracy.
3. **R-squared (R2)**:
    - Random Forest Regression has a higher R-squared value (0.312) compared to Linear Regression (0.201). A higher R-squared indicates that more variance in salaries is explained by the model, showing better overall fit.
4. **Average Yearly Salary**:
    - The average yearly salary prediction is slightly higher with Linear Regression ($149,713.58) compared to Random Forest Regression ($142,536.72).

Conclusion

- Random Forest Regression outperforms Linear Regression in terms of prediction accuracy (lower MAE and MSE) and model fit (higher R-squared).
- However, Linear Regression is much faster than Random Forest Regression

Resources

- Kaggle
- IBM
- GitHub